

# C++の `std::string` を C言語で再現する

# この話をきく動機

手続き型言語である**C**言語で、**OOP**の**3**原則の一つ「カプセル化」のメリットを享受できる

**C++**コンパイラがどのようにクラスという機能を提供しているか少しだけわかる

# C++クラスはtypedefした構造体で表現する

```
typedef struct {  
    char    *data;  
    size_t  length;  
    size_t  capacity;  
} String;
```

# 第一引数はクラスのポインタを受け取る (**this**にあたる)

```
int      string_construct(String *this, const char *str);  
void     string_destruct(String *this);  
size_t   string_length(const String *this);  
char     string_at(const String *this, size_t pos);
```

第一引数はクラスのポインタを受け取る (thisにあたる)

# 呼び出し側

C++

```
std::string s("hello");  
s.length(); // 5
```

C

```
String s;  
string_construct(&s, "hello");  
string_length(&s); // 5
```

# すべてのメソッドにクラス名の**prefix**をつける

```
int      string_construct(String *this, const char *str);  
void     string_destruct(String *this);  
size_t   string_length(const String *this);  
char     string_at(const String *this, size_t pos);
```

すべてのメソッドにクラス名のprefixをつける

# 呼び出し側

C++

```
std::string s("hello");  
s.length(); // 5
```

C

```
String s;  
string_construct(&s, "hello");  
string_length(&s); // 5
```

# 例外放出は**int**の返り値で表す

```
int    string_construct(String *this, const char *str);  
void    string_destruct(String *this);
```

```
String  s;  
if (string_construct(&s, "hello") < 0)  
    return (-1);  
string_destruct(&s);
```



メンバをprivateにする

これだと `length == strlen(data)` を保証できない

```
typedef struct {  
    char    *data;  
    size_t  length;  
    size_t  capacity;  
} String;
```

メンバをprivateにする

## 以下のように分ける

```
// String.h
typedef union String
{
    unsigned char    _[24];
    void             *_align_p;
    size_t           _align_s;
} String;
```

```
// StringPrivate.h
typedef struct {
    char    *data;
    size_t  length;
    size_t  capacity;
} String_impl_;
```

メンバをprivateにする

## 実装側で String\_impl\_ にキャストする

```
// String.c
size_t string_length(const String *this)
{
    const String_impl_ *impl = (const String_impl_ *)this;

    return (impl->length);
}
```

メモリアラインメントとは

## void \*\_align\_p と size\_t \_align\_s って？

```
// String.h
typedef union String
{
    unsigned char    _[24];
    void             *_align_p;
    size_t           _align_s;
} String;
```

**CPU**は「型のサイズの倍数のアドレス」にデータが置かれている前提で読み書きする

—8バイトのポインタなら**8**の倍数のアドレスに置く必要がある

—守らないと速度低下、環境によってはクラッシュ (**SIGBUS**)

—普段はコンパイラが型を見て自動で調整してくれる

メモリアラインメントとは

## `unsigned char _[24]` だけだと 「**1**バイト境界でよい」と判断されてしまう

- 中身の正体は `char *` と `size_t`、本当は**8**バイト境界が必要
- そこで union に `void *` と `size_t` を混ぜて「**8**バイト境界に置け」と伝える
- サイズは**24**バイトのまま、境界の要求だけ8バイトに引き上がる

# カプセル化のメリットの享受

## 不正値の防止

メソッド経由からしかアクセスしない限り、メンバの状態が崩れないことが保証される

## 可読性と使いやすさ

使う側は内部の実装を知らなくていい  
→ 複数人で並列で作業できる